IT-Security

Chapter 6: Secure Software Engineering

1. Security analysis
2. Secure design
3. Security testing

Prof. Dr. Reiner Hüttl, TH Rosenheim, © 2026, 15.03.2026

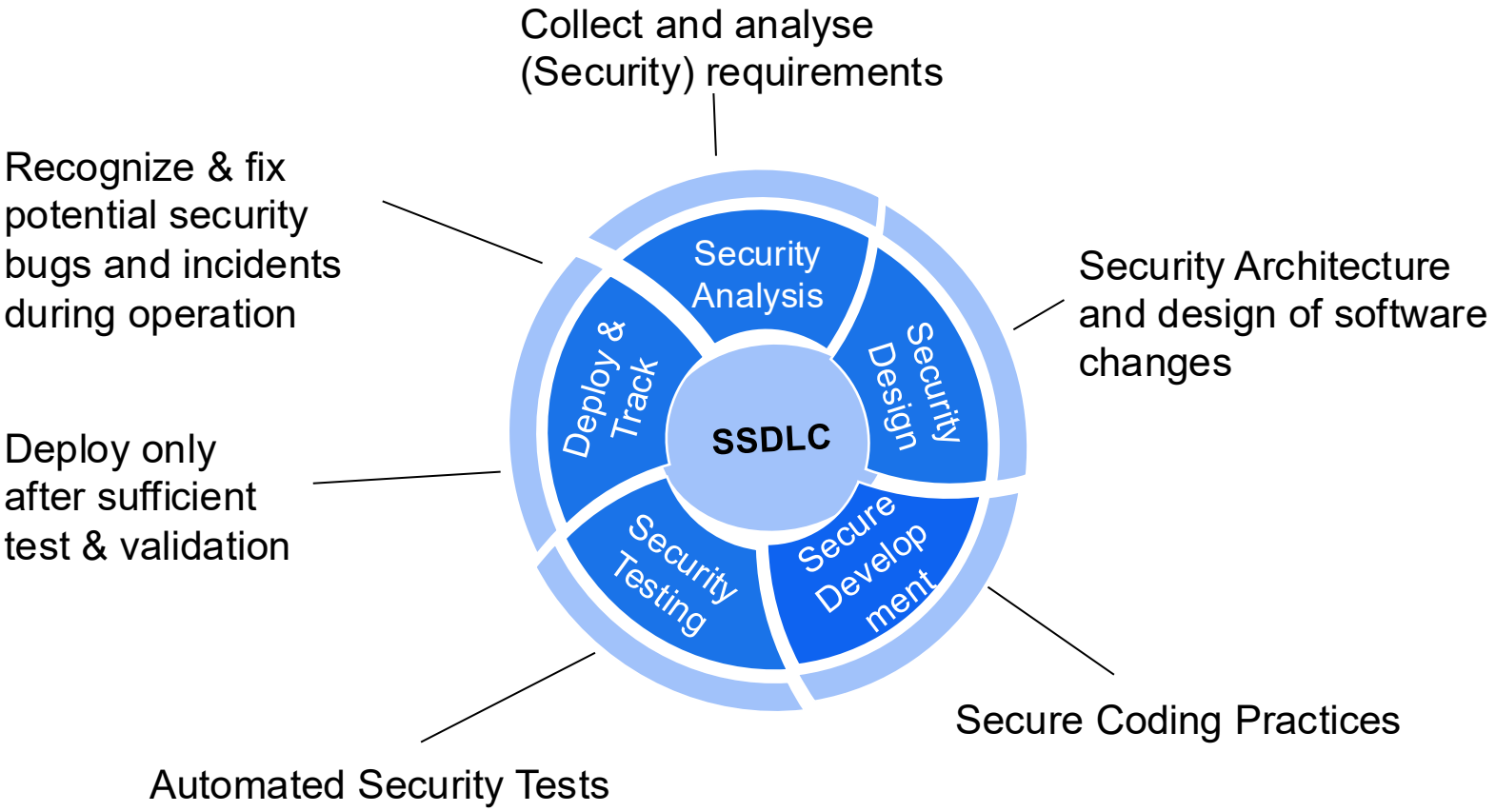
What do we want to learn?

- How do we get security into the software engineering process?
- What should we consider regarding security before implementation?
- What are design principles for security?
- How can I verify security in my IT-System?

Why Secure Software Engineering?

- Insecure software can cause nasty surprises

Secure Software Development LifeCycle

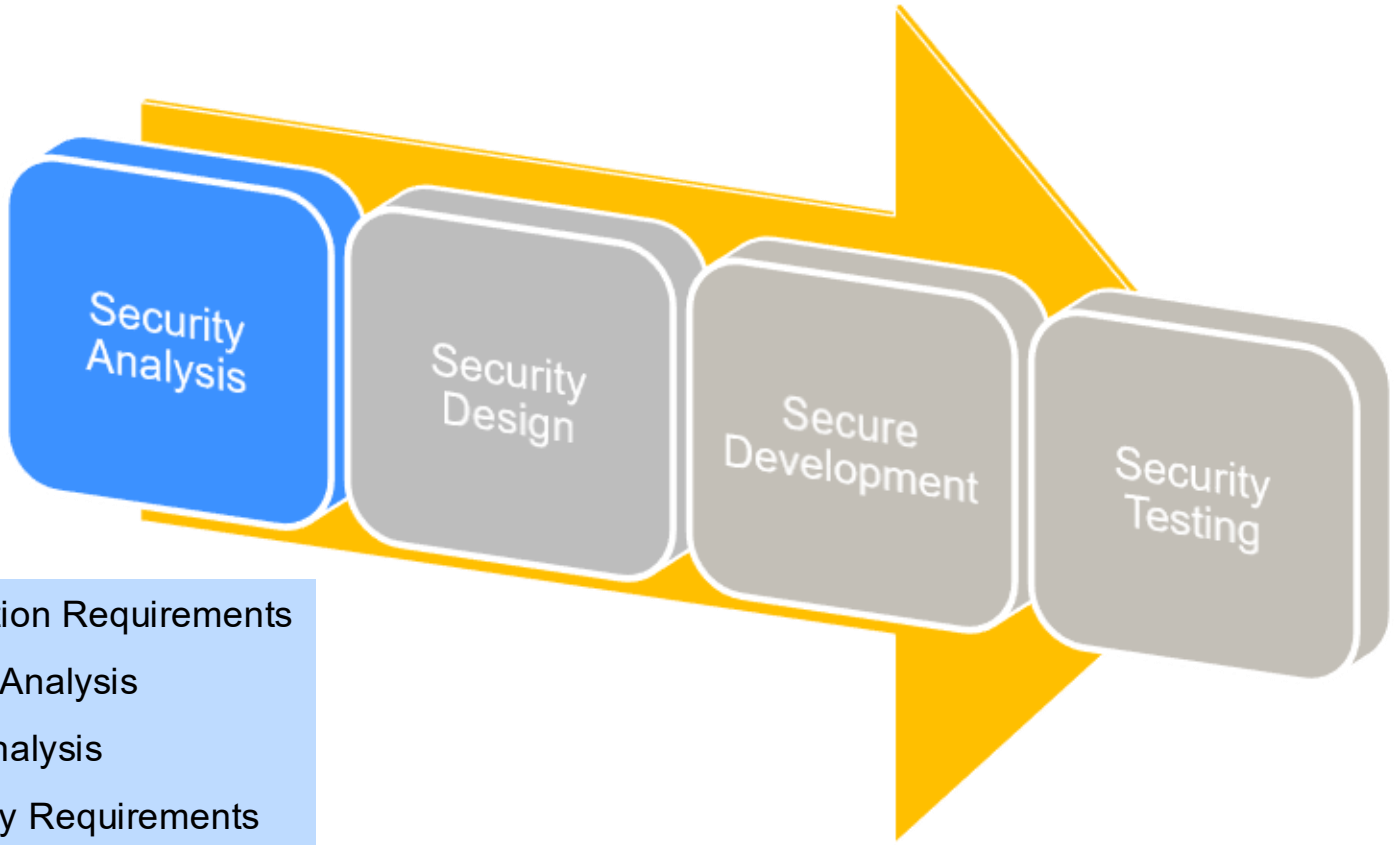


Technische Hochschule Rosenheim

Chapter 6.1: Security Analysis

- Protection requirements
- Threat analysis
- Risk analysis

Security Analysis



- Protection Requirements
- Threat Analysis
- Risk Analysis
- Security Requirements

Specify the protection requirements

- Identify critical **information objects**
  - Evaluation regarding security objectives (CIA)
  - What is the damage if security objectives are violated?
- Evaluate the protection requirements of the **use cases**
  - Which use cases cause high damage if security objectives are violated?
  - Also consider technical use cases (e.g., certificate management, system administration, authorization assignment).
- Assign **roles and rights** in the system
  - Which users/roles are there?
  - Who is allowed to do what?
  - Establish access control principles ("need to know", "segregation of duties", ...)
- Identify and analyze **threats**
  - Threat Modeling
  - Risk analysis

A modell for Threat Analysis

- **Microsoft Threat Model: STRIDE**
  - **S**poofing
    - Users should not be able to become any other user or assume the attributes of another user
  - **T**ampering
    - Data tampering involves the malicious modification of persistent data and data over networks.
  - **R**epudiation
    - Users may dispute transactions if there is insufficient auditing or recordkeeping of their activity

https://docs.microsoft.com/en-us/previous-versions/commerce-server/ee823878(v=cs.20)

STRIDE

Microsoft Threat Model: STRIDE

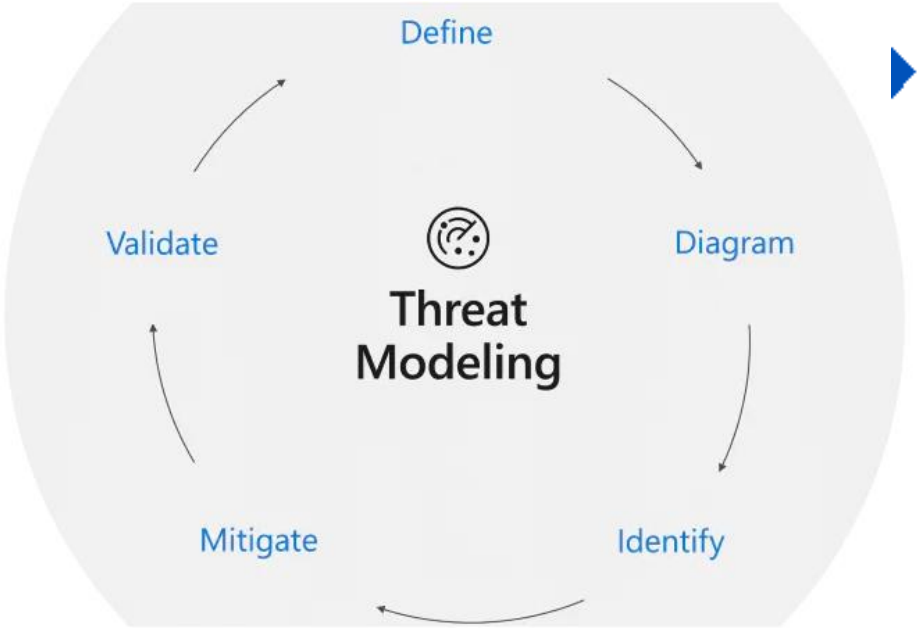
- **I**nformation Disclosure
  - The exposure of information to individuals who are not supposed to have access to it
- **D**enial of Service
  - Deny service to valid users—for example, by making a Web server temporarily unavailable or unusable

  .
- **E**levation of Privilege
  - An unprivileged user gains privileged access and thereby has sufficient access to compromise or destroy the entire system

Mapping of STRIDE to CIA-AN

STRIDE Threat	Security Goal CIA-AN
Spoofing	**A**uthenticity
Tampering	**I**ntegrity
Repudiation	**N**on-Repudiation
Information Disclosure	**C**onfidentiality
Denial of Service	**A**vailability
Elevation of privilege	**A**uthorization

Microsoft Threat Modeling



- There are five major threat modelling steps
  - defining security requirements
  - creating an application diagram
  - identifying threats
  - mitigating threats
  - validating that threats have been mitigated

Quelle: https://www.microsoft.com/en-us/securityengineering/sdl/threatmodeling

Microsoft Thread Modeling Tool https://aka.ms/threatmodelingtool

- Alternative approaches to threat analysis
  - Misuse cases
  - Attack trees
  - Threat catalogs

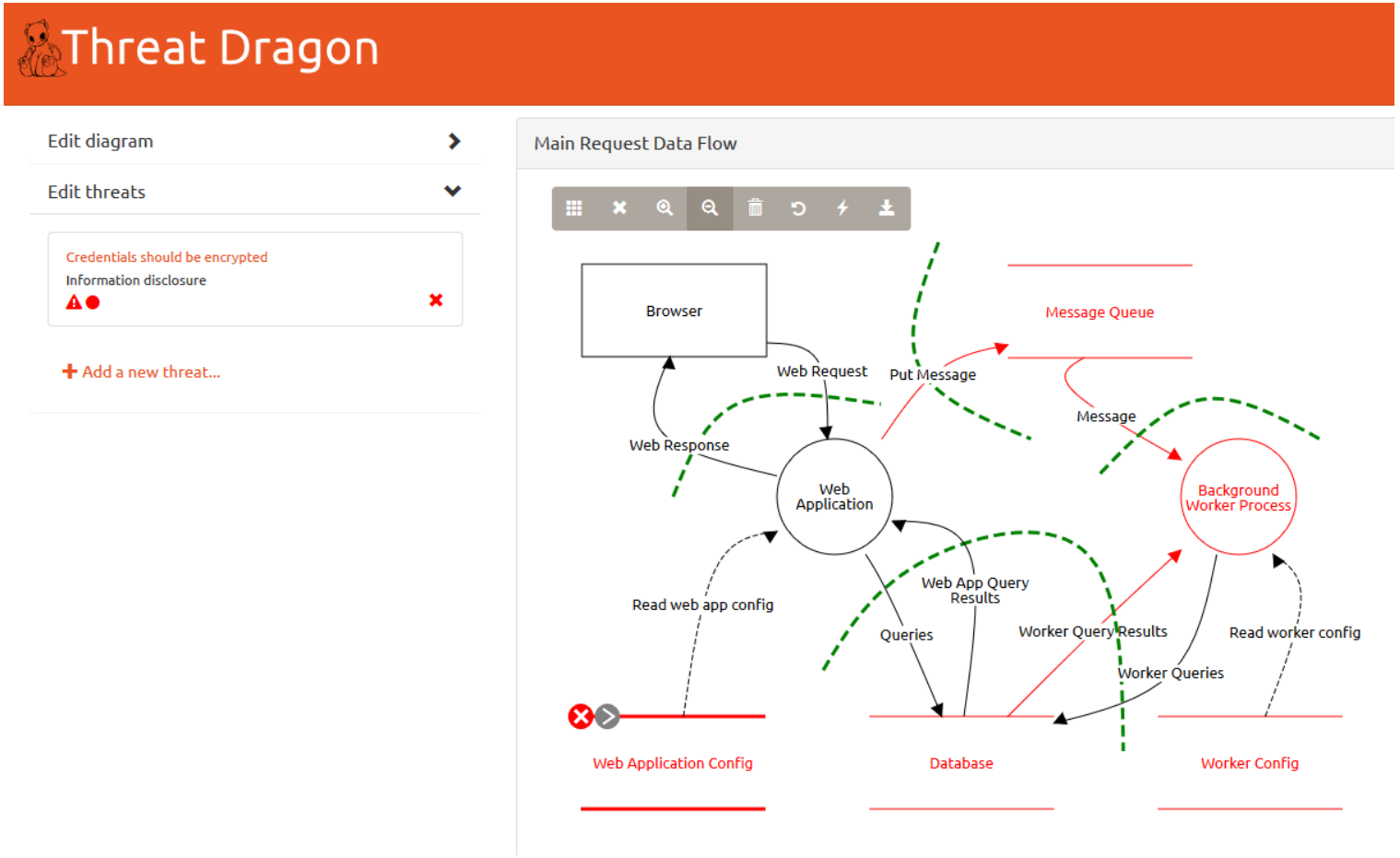
Weitere Informationen in:
Matthias Rohr: Sicherheit von Webanwendungen in der Praxis, Springer Vieweg, 2018 **(E-Book)**

Threat Modeling Process

https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html

- Decompose and model the system
  - Create an application diagram (processes, data store, actors)
  - Describe data flow (data in transit and at rest)
  - Define trust boundaries
- Identify Threats
  - Define all possible threats
  - Identify attack vectors, attack trees and misuse cases
  - Map threat agents to application entry points
  - Define the impact and probability for each threat
    → **Risk Analysis**
- Determine Countermeasures and mitigation
  - Identify risk owner (responsible for mitigation)
  - Build risk treatment strategy (Reduce, Transfer, Avoid, Accept)

Sample Model from modeling tool OWASP Threat Dragon



https://owasp.org/www-project-threat-dragon/

Example for Threat Modeling with STRIDE: Identify threats and assign type and mitigations

Threat	Type	Mitigation
Unauthorized request to DB	I	All queries to be authenticated
DB Credential Theft	I	Use FW to restrict access to DB to only background Worker IP
Message Tampering in Message queue	T	Sign all messages
Fake massages in queue	S	Implement authentication on queue
Generate malicious messages that Background Worker cannot process	D	Validate content of messages before processing, reject messages with invalid content, log the rejection, do not log the malicious content
Brute forcing of Web Application Login	E	Slowdown login attempt after unsuccessful login, 2FA for admin accounts
Sniffing of Web requests	I	Https Encryption of all requests
SQL injection	T, I	Input validation
Undocumented change of Web App Config	R	Auditing all changes in Web App Config, access control to Web App Config

Risk Analysis

- A comprehensive risk/threat analysis is costly and time-consuming
  - often the customer/client is not ready for it
  - → Perform a pragmatic risk analysis
- Focus on the most important risks
- Focus on data criticality and interfaces
- Risks must be assessed by the responsible parties (ISO, DPO, product owner, management)
- Establish transparency about the assessment of risks
  - Review by ISO/DPO

Risk analysis for Logging

Logstash **Kibana**

- We consider logging in a cloud application as an example
- Security Goals

  - The root cause of incidents or faulty platform or application behavior can be adequately analyzed and identified.
  - Required log data and analysis tools are available and correspond to the actual state of the system at the relevant time.
  - The technical logs are secured from unauthorized access and manipulation.

Risks at Logging

- **R-1: Missing log data.** An incident cannot be sufficiently analyzed because relevant log information for the required period of time has not been collected, e.g. due to a misconfiguration/failure of the log stack or according infrastructure components.

Availability

- **R-2: Loss of log data.** Log information gets lost, e.g. due to a failure of the log storage.

Integrity

- **R-3: Manipulation of logs.** The root cause of an incident can be hidden or obscured by modification or deletion of log data.

Availability

- **R-4: No access to log data.** Relevant log data cannot be viewed when required due to blocked access, e.g. missing credentials

Confidentiality

- **R-5: Disclosure of sensitive log information.** Information written to log files can give valuable guidance to an attacker or expose sensitive user data

Compliance

- **R-6: Violation of deletion obligation.** To store log files longer than the allowed retention period violates compliance (e.g. GDPR)

Security Requirements for Logging

Requirements describe what a system must do or what goals are to be achieved.
They are abstract and mostly **result-oriented**, **not solution-oriented**.

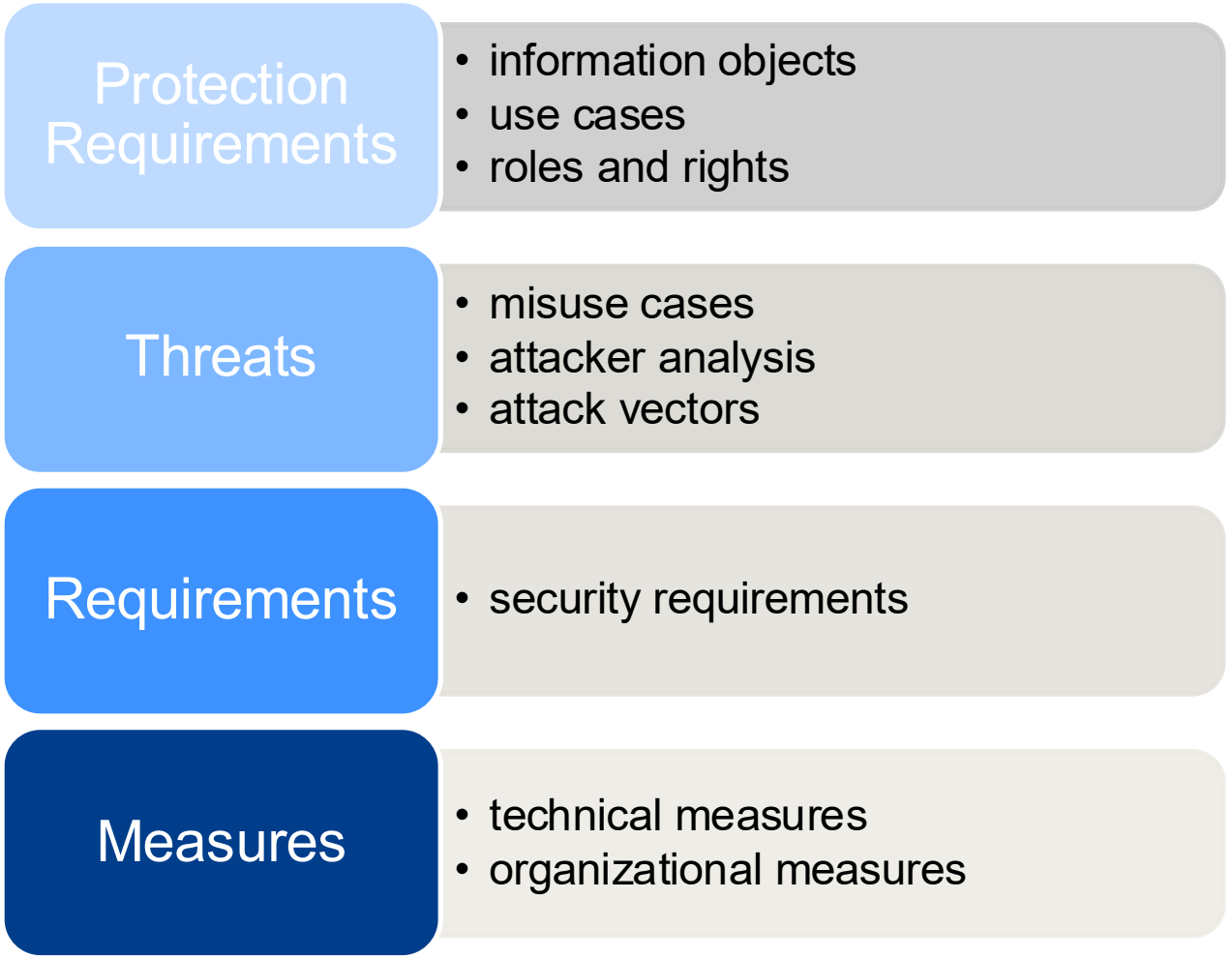
System Component	Risk	Risk name	Security Requirement
Logging	R-1	Missing log data	All security relevant events must be logged
Logging	R-2	Loss of log data	Logging data must not be lost even in the event of system errors or human error.
Logging	R-3	Manipulation of logs	The logging data must be protected against manipulation (e.g. by attackers, inside offenders, technical errors)
Logging	R-4	No access to log data	- An access concept to the logging data must be implemented. - The logging data must be accessible to administrators in the event of system failures.
Logging	R-5	Disclosure of sensitive log information	Only authorized persons may access the logging data for defined purposes. Logging data must be stored in a secure area. Logging data must not be copied and distributed. Only log sensitive information if it is required for troubleshooting.
Logging	R-6	Violation of deletion obligation	The logging data must be deleted at the end of the retention period (e.g. three months) due to the GDPR.

Security Measures for Logging

Security measures describe how requirements are implemented technically or organizationally. They are specific and **solution-oriented**

System Component	Risk	Risk name	Mitigating measures
Logging	R-1	Missing log data	- regular review of log data for their correctness - perform error tests with logging data control
Logging	R-2	Loss of log data	- backup of log data - monitoring of logging with alerting in case of failure
Logging	R-3	Manipulation of logs	- secure log data by cryptographic checksums - strict access control to log data - audit the access to log data
Logging	R-4	No access to log data	- Special admin access to logging data, - Guarantee availability via cloud provider
Logging	R-5	Disclosure of sensitive log information	- isolation of log data from the rest of the system - Strict role-based access control to log data - encryption of data at rest
Logging	R-6	Violation of deletion obligation	- automatic deletion of log data immediately after end of retention period - no local copies / snapshots of log data

The result of the Security Analysis is a Security Concept

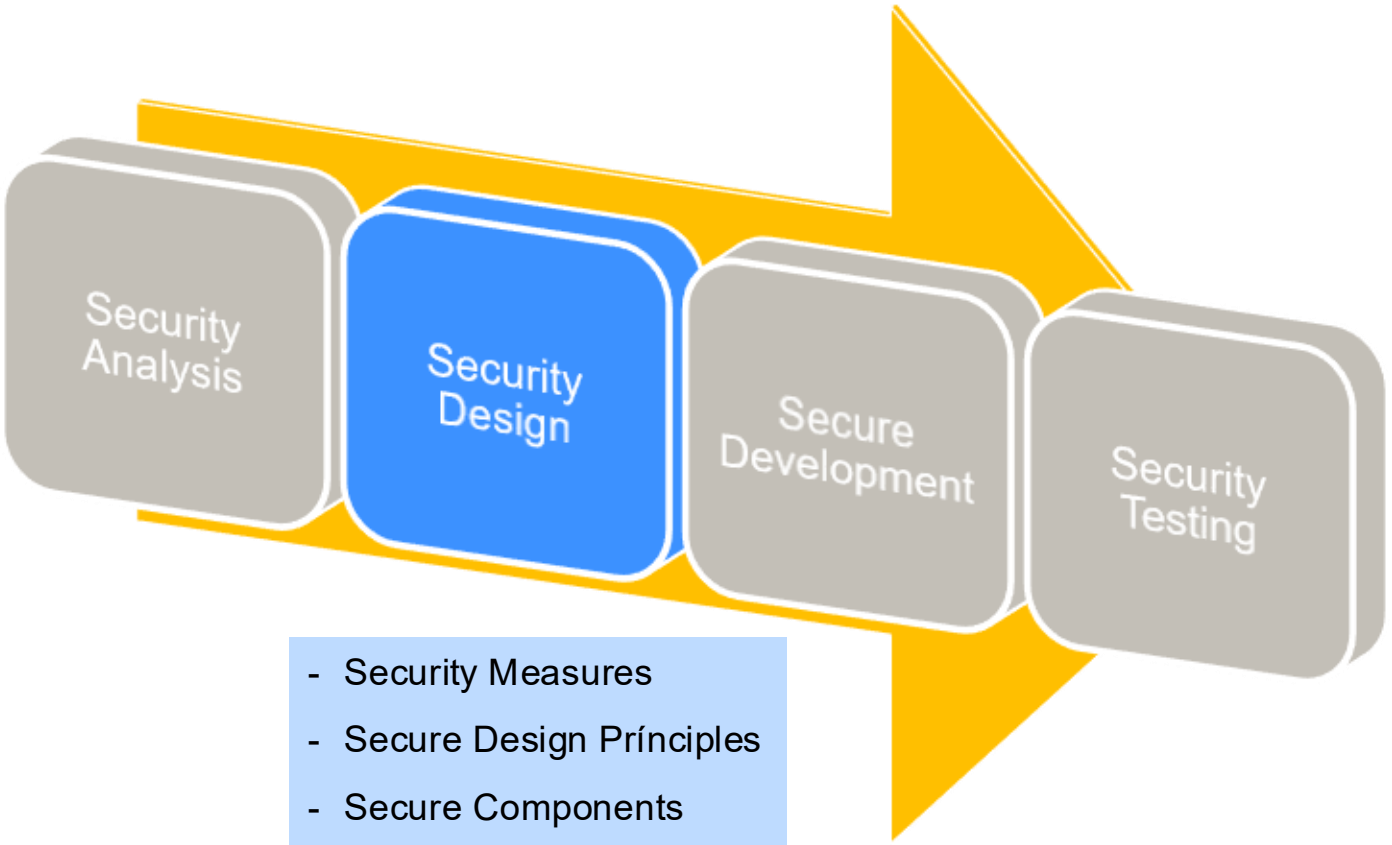


Technische Hochschule Rosenheim

Chapter 6.2: Secure Design

- Secure Design Principles
- Secure Components

Security Architecture and -Design



- Security Measures
- Secure Design Prínciples
- Secure Components
- Secure Channels

Secure Design Principles

OWASP Nr. 6:
Insecure Design

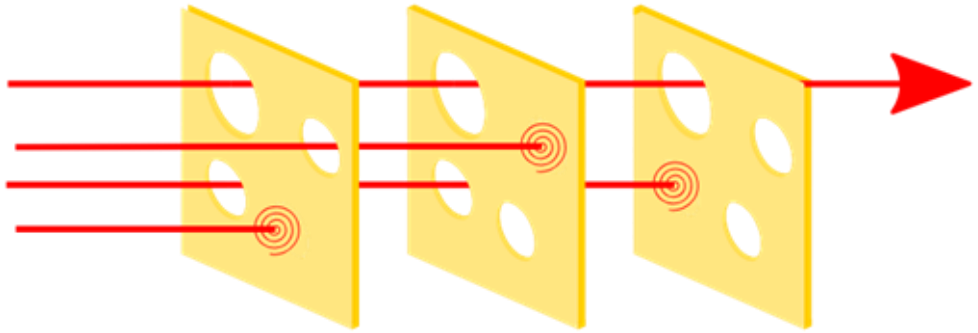
- In the design phase there are a lot of general principles that lead to secure software systems.
- Many of these principles should also be considered in the other development phases
- There are many sources and principles
  - OWASP https://cheatsheetseries.owasp.org/cheatsheets/Secure_Product_Design_Cheat_Sheet.html
  - NCSC https://www.ncsc.gov.uk/collection/cyber-security-design-principles
  - OSA http://www.opensecurityarchitecture.org/cms/foundations/design-principles
- There is even a generic vulnerability on this topic
  - CWE-657: Violation of Secure Design Principles
    https://cwe.mitre.org/data/definitions/657.html
  - and a **OWASP Top 10: A06:2025 Insecure Design**
    https://owasp.org/www-project-top-ten/

Secure Design Principles

1. Zero Trust
2. Keep it simple
3. Secure Defaults
4. Open design
5. Fail secure, Fail safe
6. Minimize attack surface
7. Psychological acceptability

Defense-in-Depth

- Multiple layers of Security
  - no measure offers absolute protection
  - an existing protective measure does not render other measures obsolete
  - If possible, several safety measures should be used to counter threats with different techniques.
  - Assumption: one of our security levels is compromised.
    How can we still protect our system?”



Swiss Cheese Model

Divide your architecture in reliable and secure components

- Ensures that only clean messages enter the components
- Here typical measures to protect the security objectives are:

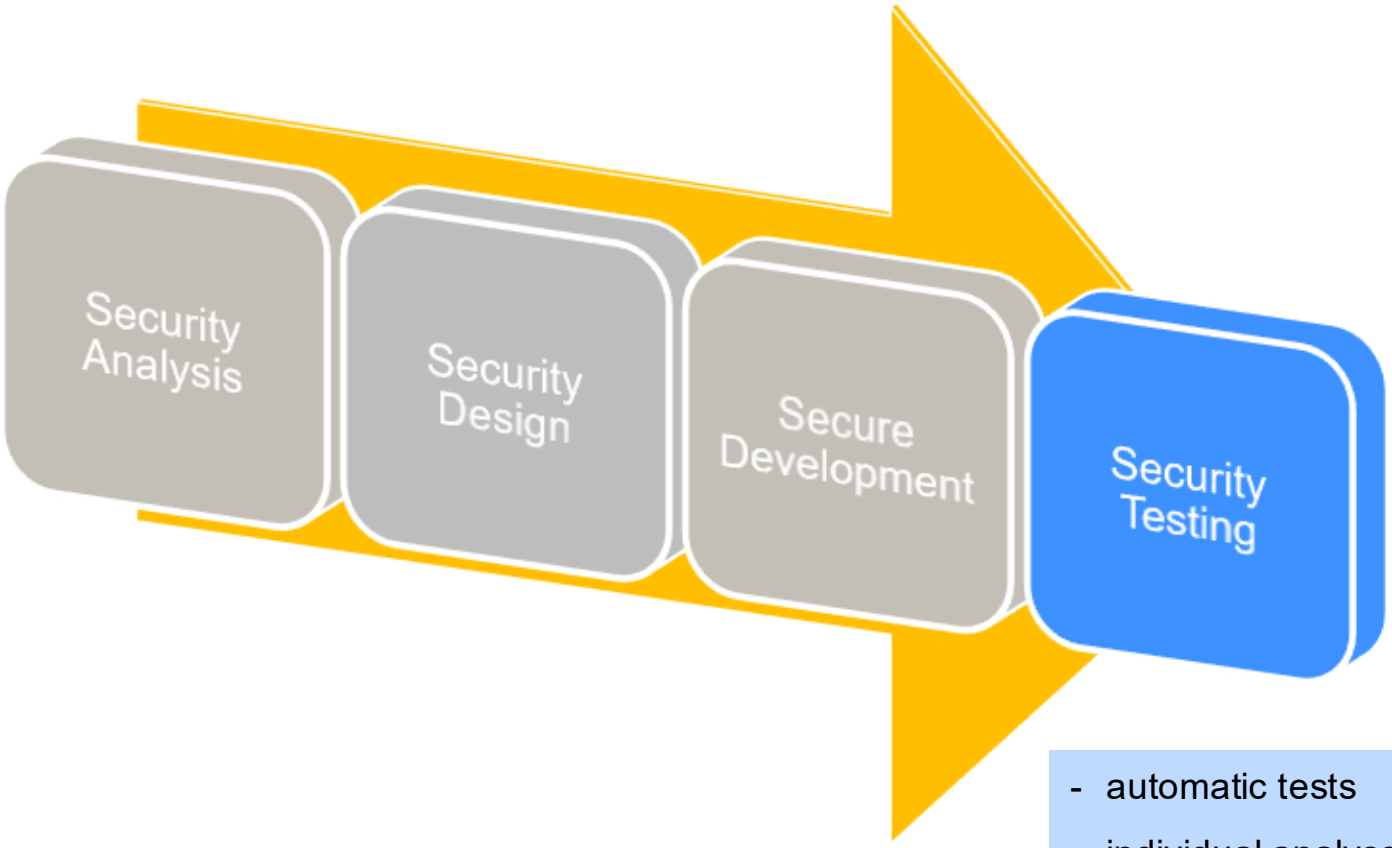
Security goal	Task / Requirement	Possible measures
Confidentiality	accept only secure channel	• HTTPS with Perfect Forward Secrecy • encryption on application level
-- " --	check whether the sender is allowed to send the message	• check URLs against ACL (.htaccess) • application level: check content of message
Authenticity	check authenticity of sender	authentication (e.g. with client certificates)
-- " --	check actuality (Freshness) of the message	assign nonces (one-time IDs) in the response and check them in the request
Integrity	check content of the message and reject if necessary	validate semantically (application-specific) and syntactically (cryptographic checksums)
Availability	detect and defend DoS-attacks	**rate limiting** and **tarpits**.
Non-Repudiation	store the contents of a message sealed	signature and signature-verification

Technische Hochschule Rosenheim

Chapter 6.3: Security Testing

- SAST
- DAST
- Dependency check

Security Testing



- automatic tests
- individual analyses

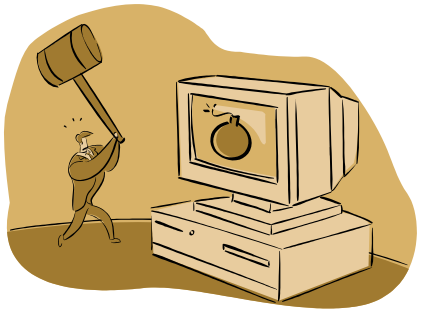
Software development is done on a software assembly pipeline

- Product quality is determined automatically

ubiquitous feedback devices

Measure for secure software: Test all software!!!

- Permanent manual and automatic tests are necessary to detect and eliminate errors in the software in time
- The types of testing procedures is diverse, here are a few examples:
  - unit test, component test, integration test, system test
  - acceptance test, Test Driven Development
  - usability test
  - regression test
  - load test, robustness test



Security Testing – Automated Testing

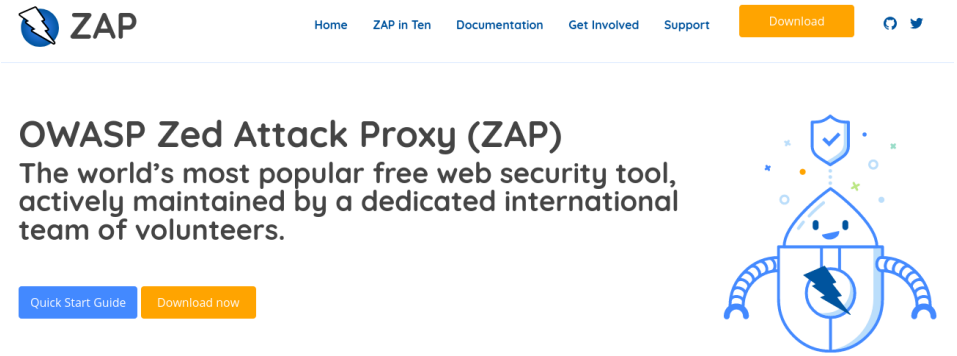
- **SAST Static Application Security Testing**
  - automated static analysis tools
  - provide warnings at compile time where bugs can occur
  - examples: SonarQube, FindBugs, Fortify
- **DAST Dynamic Application Security Testing**
  - Web Security Scanner, Vulnerability Scanner: Computer programs that scan target system for known vulnerabilities, e.g. OWASP ZAP, Nessus, Qualys
  - **Passive Scans**
    - scan the data between browser and web server for patterns
    - do not change the request or the response
    - takes place in the background
  - **Active Scans**
    - search vulnerabilities by applying known attacks
    - **should not be applied to third party web applications without permission**

Individual analysis methods

- **Penetration test**
  - testing of all system components with means and methods that an attacker (hacker) would use
  - black box, white box test
  - must be planned (scoping, preparation, pre-analysis, analysis, debriefing, retest)
  - must be done by professionals
- **Fuzz Testing (Fault Injection)**
  - generates (semi) automated invalid, unexpected or random input data
  - monitors exceptions, crashes or missing error handling (monitoring)
- **Security Code Review**
  - mutual reviews are necessary in addition to SAST

Security testing tools

- Attack Proxies – tools for dynamic analysis
  - Nessus, Qualys, Burp Suite
  - OWASP ZAP



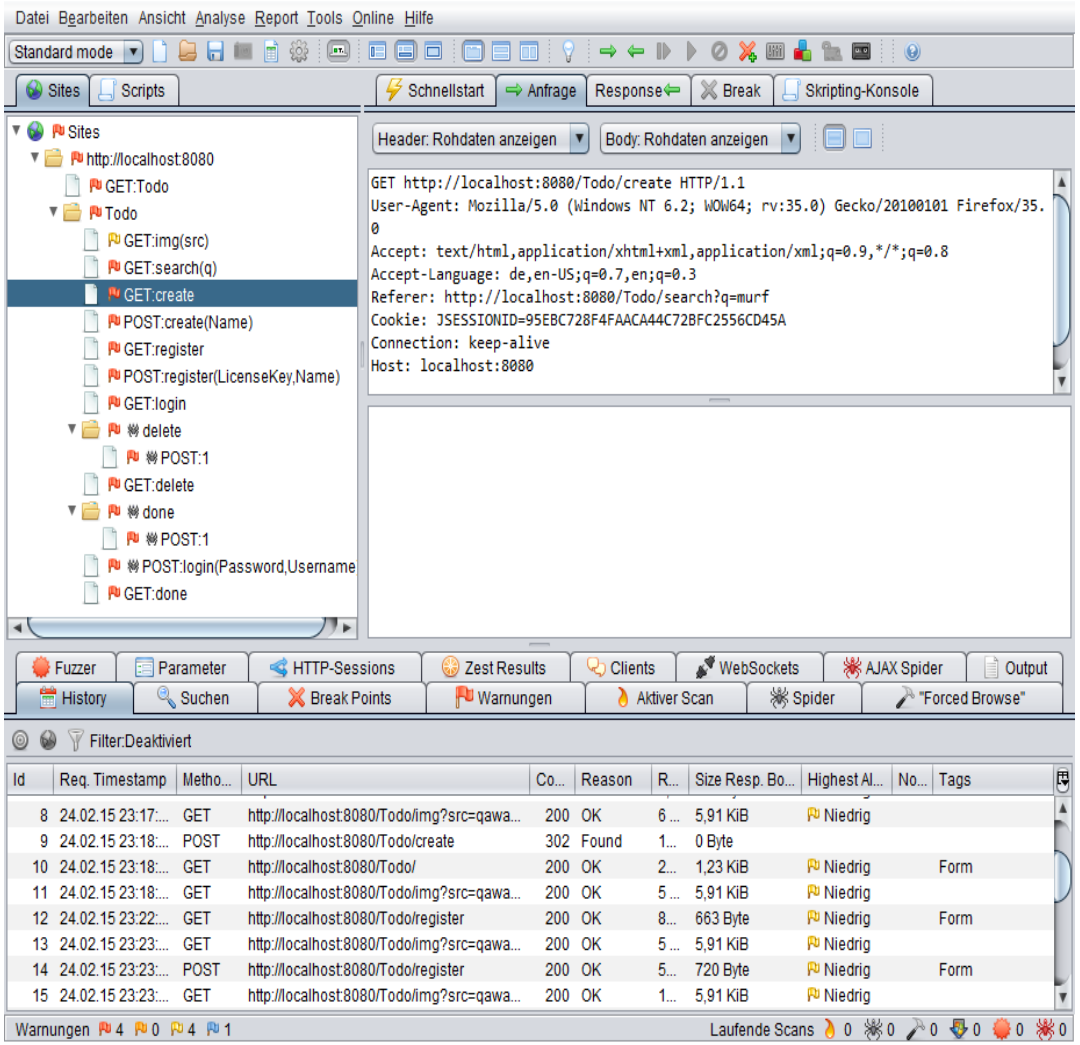
https://www.zaproxy.org/

- OWASP Dependency Check
  - Software Composition Analysis (SCA) tool that attempts to detect public disclosed vulnerabilities contained within a project's dependencies

OWASP ZAP - ZAP Attack Proxy

https://www.zaproxy.org

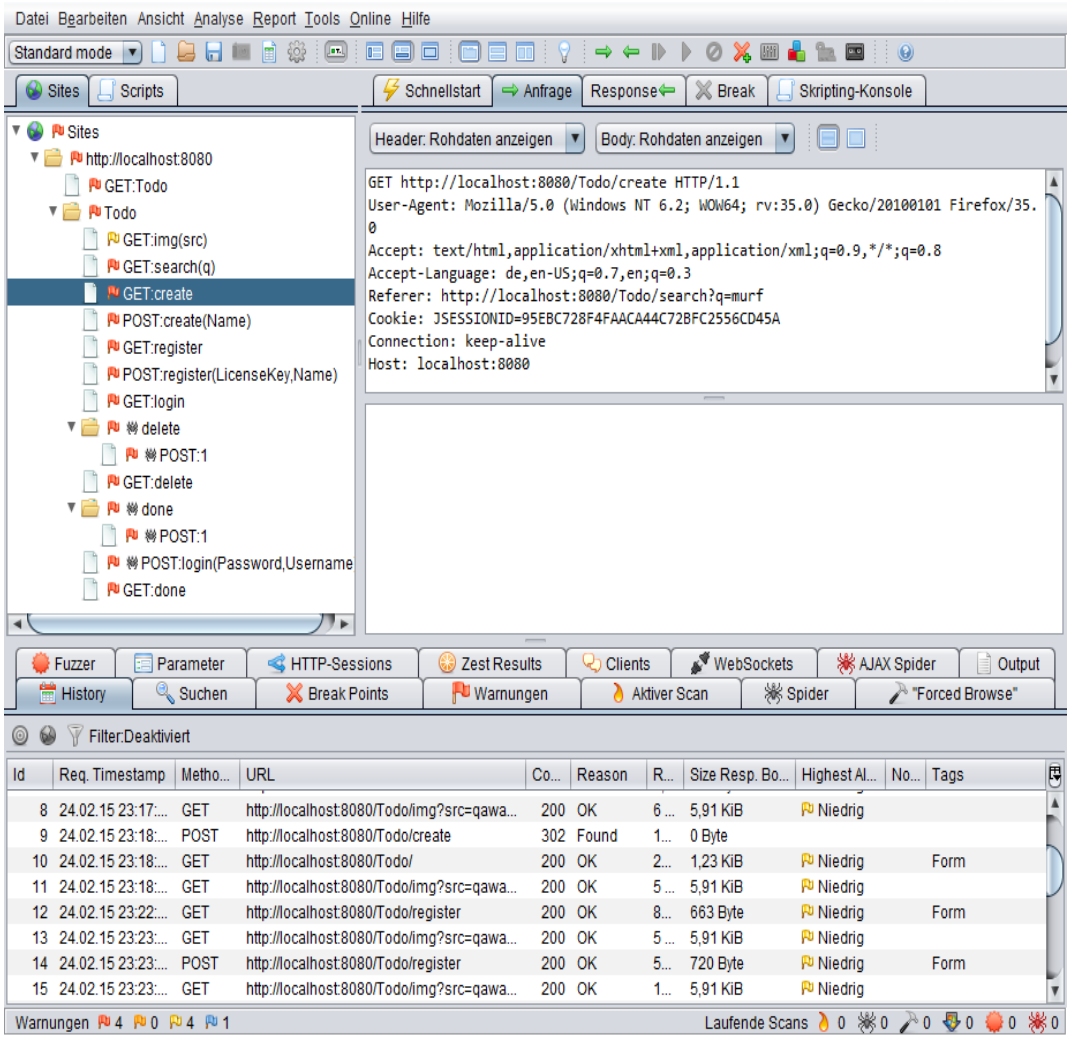
- Open-source Java tool with very active community
- Attack proxies are a standard tool for pen testers
- ZAP is explicitly not only for Pen Tester but also for developers
- Extendable by plugins



OWASP ZAP - ZAP Attack Proxy

Features:

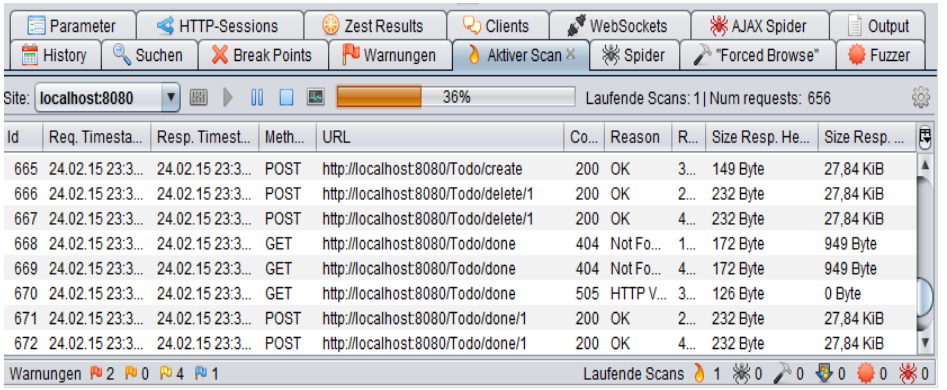
- Intercepting Proxy
  - switches between browser and server and records requests
  - on-the-fly adjustment of request/response by breakpoints
- Spider
  - automatic detection of new URLs by parsing the responses

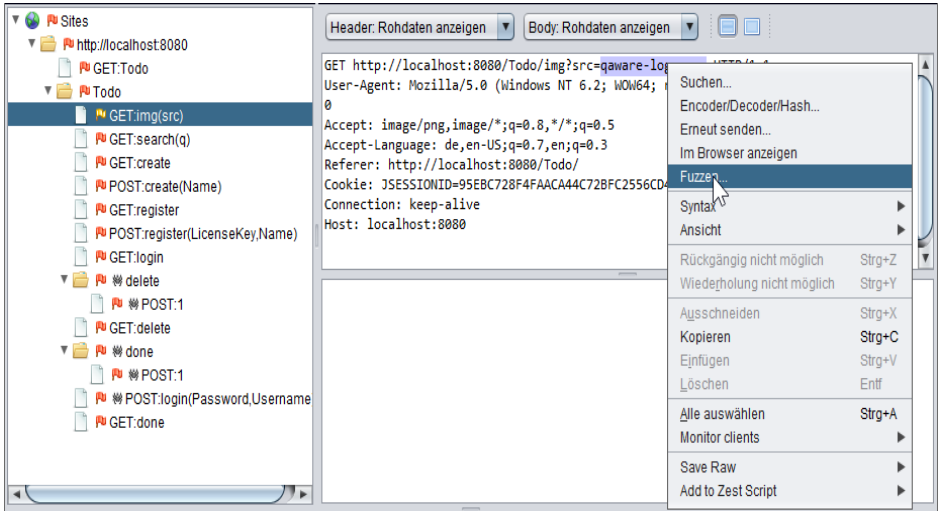


OWASP ZAP - ZAP Attack Proxy

Features II:

- Active and passive scanner
  - Passive scan: Scans all responses for vulnerabilities
  - Active scan: Fully automatic scan of the application
- Fuzzer
  - Replaces strings in requests with known attack patterns





OWASP ZAP – Warning!

- Run scans only on systems when you have **explicit permission** from the owner
- Run active scans **only on test systems**

Hacker paragraph in the code of german law

- **§ 202c Vorbereiten des Ausspähens und Abfangens von Daten**
  https://www.gesetze-im-internet.de/stgb/__202c.html

  Wer eine Straftat nach § 202a oder § 202b vorbereitet, indem er

  1. Passwörter oder sonstige Sicherungscodes, die den Zugang zu Daten (§ 202a Abs. 2) ermöglichen, oder
  2. Computerprogramme, deren Zweck die Begehung einer solchen Tat ist,

  herstellt, sich oder einem anderen verschafft, verkauft, einem anderen überlässt, verbreitet oder sonst zugänglich macht, wird mit Freiheitsstrafe bis zu einem Jahr oder mit Geldstrafe bestraft.

  - **§ 202a** Ausspähen von Daten (data spying)
  - **§ 202b** Abfangen von Daten (interception of data, phishing)

- Preparation and possession of hacking tools is already punishable!

- In the EU: **Directive 2013/40/EU on attacks against information systems**,

OWASP TOP 3: Software Supply Chain Failures

- Problem:
  - Software is built automatically in a supply chain (**CI/CD pipeline**)
    - Often the supply chain has weaker security than the system it builds
    - The Software Bill of Materials (**SBOM**) is not managed
  - Modern software is including external libraries and frameworks
  - Some frameworks address security
    - Prepared statements in JDBC / Hibernate
    - OAuth with SpringBoot
    - Access token assignment through web container
- What if one of these frameworks has a security vulnerability?

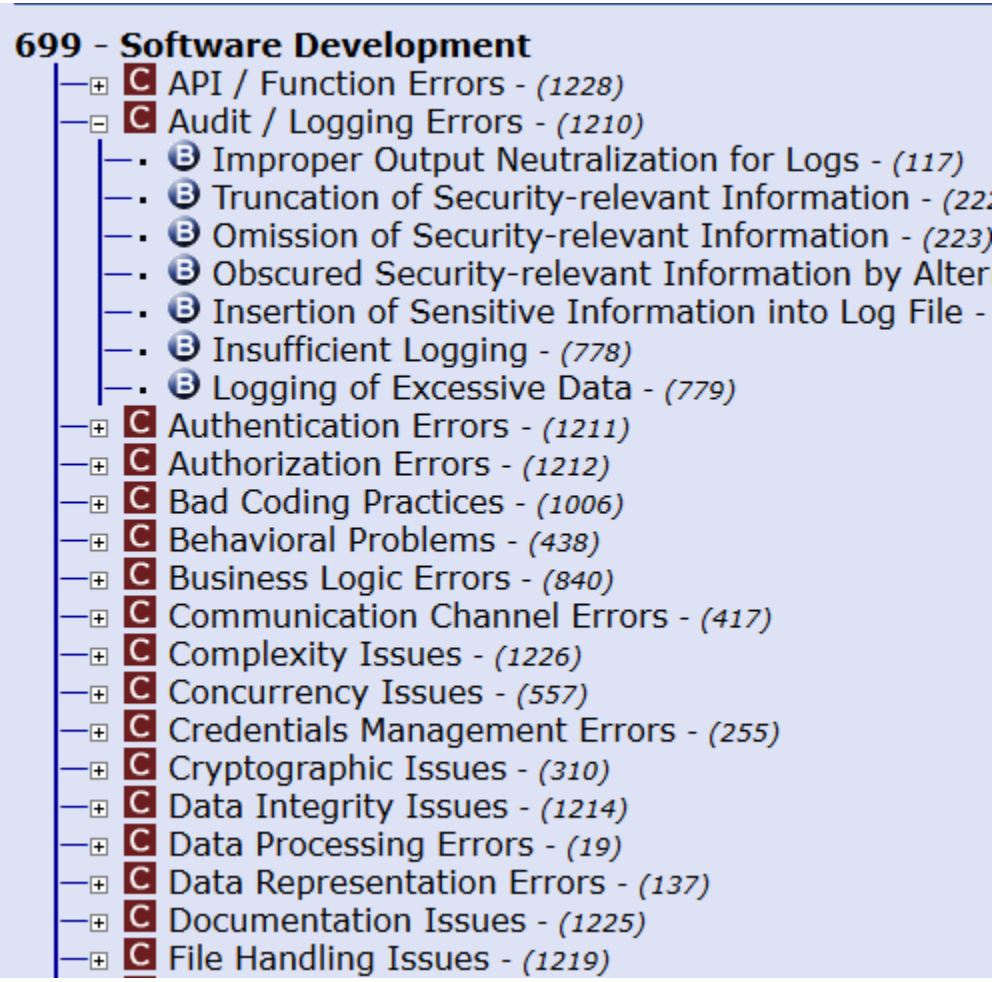
CWE / CVE – A database for known vulnerabilities

- Operated by Mitre.org
  - Non-profit-organization from the USA

- **CWE** - Common Weakness Enumeration
  - Collection of generic vulnerabilities
- **CVE** – Common Vulnerability Enumeration
  - Collection of product-specific vulnerabilities
- **CVSS** – Common Vulnerability Scoring System
  - Scoring scheme to rate the severity of a vulnerability

CWE - Common Weakness Enumeration

- Collection of generic vulnerabilities
  - product-independent
- http://cwe.mitre.org
- Important information:
  - Examples
  - Relationships - Interrelationships with other vulnerabilities
  - References - Further literature about a vulnerability
  - Detection Mechanisms - Measures how the vulnerability can be detected
  - Mitigations - Measures to prevent the vulnerability



https://cwe.mitre.org/data/definitions/699.html

The CWE – Top 25 2025

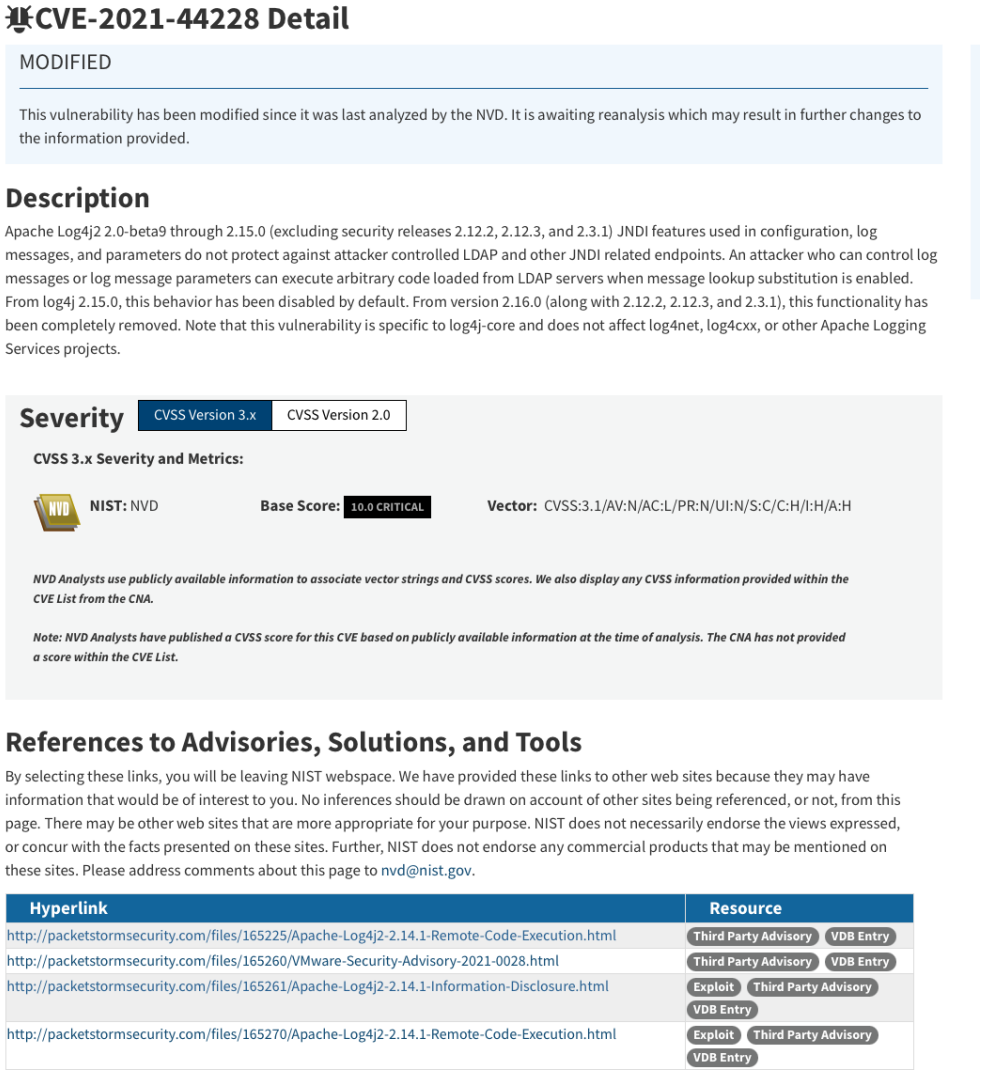
2025 CWE Top 25

Rank	ID	Name	Score	CVEs in KEV	Rank Change vs. 2024
1	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')	60.38	7	0
2	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')	28.72	4	+1
3	CWE-352	Cross-Site Request Forgery (CSRF)	13.64	0	+1
4	CWE-862	Missing Authorization	13.28	0	+5
5	CWE-787	Out-of-bounds Write	12.68	12	-3
6	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')	8.99	10	-1
7	CWE-416	Use After Free	8.47	14	+1
8	CWE-125	Out-of-bounds Read	7.88	3	-2
9	CWE-78	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')	7.85	20	-2
10	CWE-94	Improper Control of Generation of Code ('Code Injection')	7.57	7	+1
11	CWE-120	Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')	6.96	0	N/A
12	CWE-434	Unrestricted Upload of File with Dangerous Type	6.87	4	-2
13	CWE-476	NULL Pointer Dereference	6.41	0	+8
14	CWE-121	Stack-based Buffer Overflow	5.75	4	N/A
15	CWE-502	Deserialization of Untrusted Data	5.23	11	+1
16	CWE-122	Heap-based Buffer Overflow	5.21	6	N/A
17	CWE-863	Incorrect Authorization	4.14	4	+1
18	CWE-20	Improper Input Validation	4.09	2	-6
19	CWE-284	Improper Access Control	4.07	1	N/A
20	CWE-200	Exposure of Sensitive Information to an Unauthorized Actor	4.01	1	-3
21	CWE-306	Missing Authentication for Critical Function	3.47	11	+4
22	CWE-918	Server-Side Request Forgery (SSRF)	3.36	0	-3
23	CWE-77	Improper Neutralization of Special Elements used in a Command ('Command Injection')	3.15	2	-10
24	CWE-639	Authorization Bypass Through User-Controlled Key	2.62	0	+6
25	CWE-770	Allocation of Resources Without Limits or Throttling	2.54	0	+1

https://cwe.mitre.org/top25/index.html

CVE – Common Vulnerability Enumeration https://www.cve.org

- Collection of product-specific vulnerabilities of
  - operating systems
  - Programs
  - Languages
  - Frameworks
  - etc
- Naming scheme of entries: CVE-<Year>-<RunningNumber>.
- Note: Not all vulnerabilities are listed here, but overall most complete collection



CVE-2021-44228 Detail

MODIFIED

This vulnerability has been modified since it was last analyzed by the NVD. It is awaiting reanalysis which may result in further changes to the information provided.

Description

Apache Log4j2 2.0-beta9 through 2.15.0 (excluding security releases 2.12.2, 2.12.3, and 2.3.1) JNDI features used in configuration, log messages, and parameters do not protect against attacker controlled LDAP and other JNDI related endpoints. An attacker who can control log messages or log message parameters can execute arbitrary code loaded from LDAP servers when message lookup substitution is enabled. From log4j 2.15.0, this behavior has been disabled by default. From version 2.16.0 (along with 2.12.2, 2.12.3, and 2.3.1), this functionality has been completely removed. Note that this vulnerability is specific to log4j-core and does not affect log4net, log4cxx, or other Apache Logging Services projects.

Severity CVSS Version 3.x | CVSS Version 2.0

CVSS 3.x Severity and Metrics:

NIST: NVD Base Score: 10.0 CRITICAL Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H

NVD Analysts use publicly available information to associate vector strings and CVSS scores. We also display any CVSS information provided within the CVE List from the CNA.

Note: NVD Analysts have published a CVSS score for this CVE based on publicly available information at the time of analysis. The CNA has not provided a score within the CVE List.

References to Advisories, Solutions, and Tools

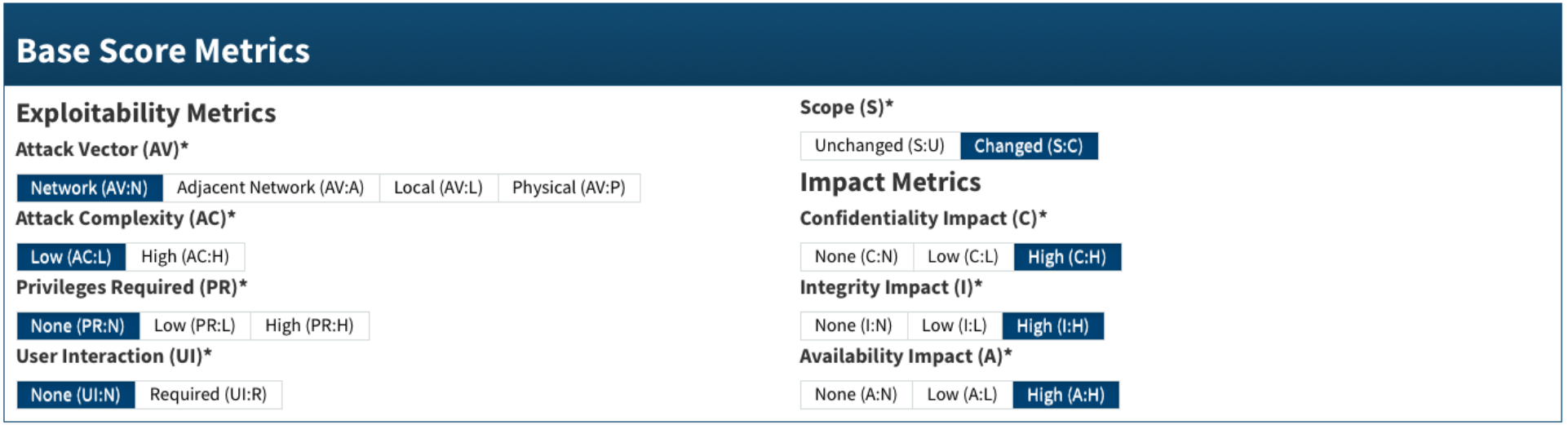
By selecting these links, you will be leaving NIST webspace. We have provided these links to other web sites because they may have information that would be of interest to you. No inferences should be drawn on account of other sites being referenced, or not, from this page. There may be other web sites that are more appropriate for your purpose. NIST does not necessarily endorse the views expressed, or concur with the facts presented on these sites. Further, NIST does not endorse any commercial products that may be mentioned on these sites. Please address comments about this page to nvd@nist.gov.

Hyperlink	Resource
http://packetstormsecurity.com/files/165225/Apache-Log4j2-2.14.1-Remote-Code-Execution.html	Third Party Advisory, VDB Entry
http://packetstormsecurity.com/files/165260/VMware-Security-Advisory-2021-0028.html	Third Party Advisory, VDB Entry
http://packetstormsecurity.com/files/165261/Apache-Log4j2-2.14.1-Information-Disclosure.html	Exploit, Third Party Advisory, VDB Entry
http://packetstormsecurity.com/files/165270/Apache-Log4j2-2.14.1-Remote-Code-Execution.html	Exploit, Third Party Advisory, VDB Entry

Log4J vulnerabilty: https://nvd.nist.gov/vuln/detail/CVE-2021-44228

CVSS – Common Vulnerability Scoring System

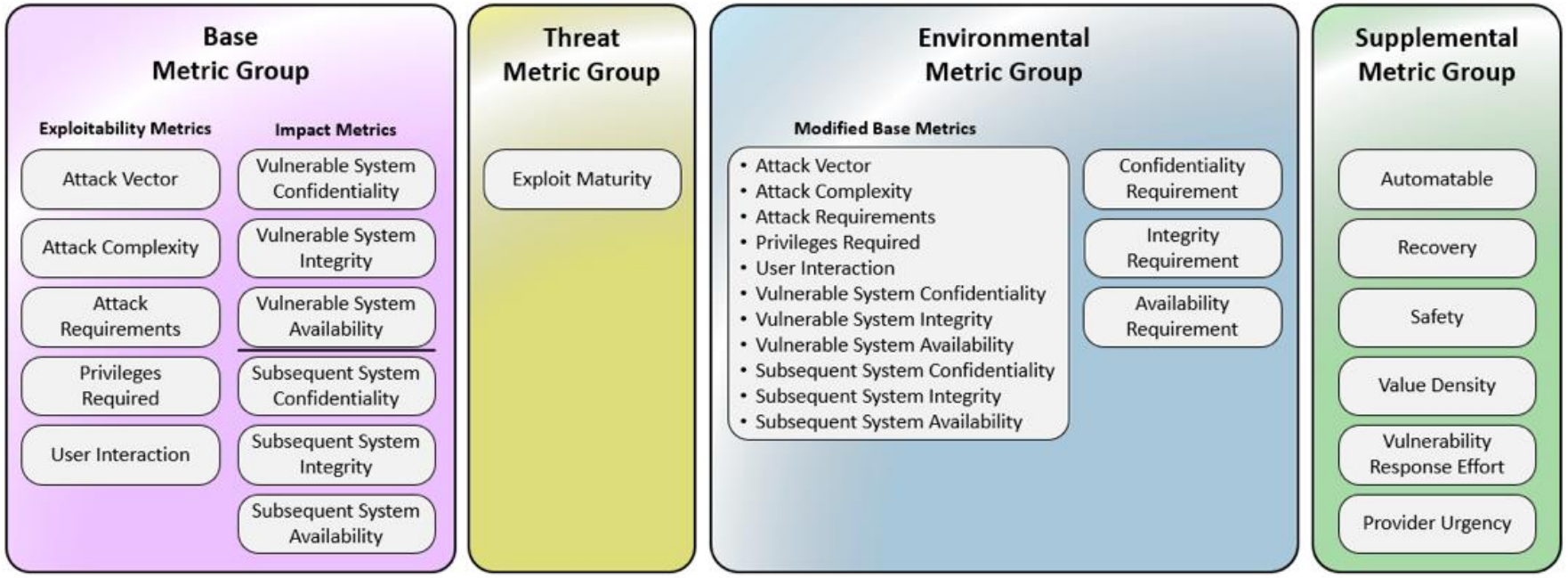
- Score from 0.0 (uncritical) to 10.0 (extremely critical)
- Attention: CVSS is a first point of reference but cannot replace an individual classification in the project context.
- Various parameters are included in the score



CVSS Version3 Scoring System Calculator

- There are websites for calculating CVSS score with detailed documentation
  https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator
  https://www.first.org/cvss/calculator/3.1

- CVSS is composed of four metric groups



https://www.first.org/cvss/v4.0/specification-document

OWASP Dependency-Check

https://owasp.org/www-project-dependency-check/

- Automated tool for analyzing the libraries in use
- Evaluates the CVEs of the National Vulnerability Database
  See http://nvd.nist.gov/

DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

How to read the report | Suppressing false positives | Getting Help: github issues

Sponsor

Project:

Scan Information (show all):

- *dependency-check version*: 6.1.6
- *Report Generated On*: Tue, 10 May 2022 22:18:33 +0200
- *Dependencies Scanned*: 271 (241 unique)
- *Vulnerable Dependencies*: 24
- *Vulnerabilities Found*: 110
- *Vulnerabilities Suppressed*: 0
- ...

Summary

Display: Showing Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
webgoat-server-8.2.2.jar: ant-1.6.5.jar	cpe:2.3:a:apache:ant:1.6.5:*:*:*:*:*:*:*	pkg:maven/ant/ant@1.6.5	MEDIUM	1	Highest	29
webgoat-server-8.2.2.jar: ant-launcher-1.6.5.jar	cpe:2.3:a:apache:ant:1.6.5:*:*:*:*:*:*:*	pkg:maven/ant/ant-launcher@1.6.5	MEDIUM	1	Low	17
webgoat-server-8.2.2.jar: challenge-8.2.2.jar: bootstrap.min.js		pkg:javascript/bootstrap@3.1.1	MEDIUM	5		3
webgoat-server-8.2.2.jar: dirgra-0.3.jar	cpe:2.3:a:jruby:jruby:0.3:*:*:*:*:*:*:*	pkg:maven/org.jruby/dirgra@0.3	MEDIUM	2	Highest	17
webgoat-server-8.2.2.jar: jackson-databind-2.11.4.jar	cpe:2.3:a:fasterxml:jackson-databind:2.11.4:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-modules-java8:2.11.4:*:*:*:*:*:*:*	pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.11.4	HIGH	1	Highest	42

OWASP Dependency-Check

Scan options
Command Line Tool
Maven Plugin
Ant Task
Gradle Plugin
Jenkins Plugin
and more

File types
Archive (.zip, .war, .ear, …)
.Net Assembly
Java archive files. .jar
JavaScript files
Node.js
Ruby files

- As with all automated tools: you must consider false positives and false negatives
  - false positives can be removed from the scan by configuration
  - pay particular attention to occurrences with 0 findings
  - manual research is worthwhile here

What to do when security vulnerabilities are found?

- Step 1 - Information Gathering:
  - Which component is affected, what is the attack vector, what are the implications, are there updates / workarounds, do exploits already exist.
  - CVE is often a good starting point for further research
- Step 2 - Evaluation:
  - How does the vulnerability affect my system security.
  - What are appropriate measures to avoid vulnerability of the system
  - Update, replace library with alternative
  - Often useful: define short-term and long-term measures
- Step 3 - Implement measures

Summary Secure Software Engineering

- The topic of security must be integrated into the software engineering process from the very beginning.
- The Secure Software Development Life Cycle continuously analyzes, implements and tests the security requirements.
- Insecure Software can cause nasty surprises